Revise Hibernate basics n Hibernate Architecture
CRUD operations in Hibernate
Persistence Context (L1 cache) working + entity state transitions
Blob Handling

Revision -

1. Name n explain important blocks of hibernate architecture
1.1 org.hibernate.Session - i/f
-Represents persistence manager
-associated with L1 cache (cache of refs) + pooled out DB
-short lived , thread un safe , no need of synchronization

1.2 org.hibernate.SessionFactory - i/f
provider of Session
openSession vs getCurrentSession - is there an existing session already bound to the current thread ?
No - creates NEW session (L1 cache + db cn pooled out)
YES - returns EXISTING session to the caller
Why it checks within a thread?
Since in hibernate.cfg.xml , prog has already added -
Property : current_session_context_class-thread
Applicable for - getCurrentSession

SF - singleton / DB , immutable , inherently thread safe , long lived
asso with L2 cache (has to be explicitly enabled)

1.3 org.hibernate.cfg.Configuration
static init block
-new Configuration().configure().buildSessionFactory() - SF

2. Have you extracted hibernate docs (java docs) - from Hibernate reference material ?
"Hibernate Reference Material\hibernate-javadocs.zip"

3. Steps
1. Copy n import test_hibernate_basic
2. Copy hibernate.cfg.xml under <resources> folder
3. Create HibernateUtils
-provide depcy(SessionFactory) for DAO

4. TestHibernate - main class

5. Create the entity (eg Employee)
ID propertry MUST be - Serializable
Tip from Gavin King - ref type (eg - Integer | Long)
This is the property exclusively used for HBM (hibernate mapping) purpose.
Annotations - @Entity , @Id -PK
@Table , @Column, @Enumerated, @GeneratedValue(strategy=GenerationType.IDENTITY) , @Lob

6. Add <maping class="F.Q entity class name"/> in hibernate.cfg.xml

7. Run TestHibernate.java again
Confirm auto table creation

8. Create DAO
i/f n imple class
CRUD method
Steps
8.1 get session from SF
8.2 begin tx
8.3 try
{

    tx.commit();
}catch (RuntimeException e)
{
  rollback tx
  throw e
}
Session API -
public Serializable save(Object o) throws HibernateException
-insert
public <T> T get(Class<T> class, Serializable id) throws HibernateException
- select

Interview / exam objective .
Assume - getCurrentSession
Q - What exactly happens upon commit ?(tx.commit)
1. session.flush();
2. Hibernate performs auto dirty checking - upon flushing the sesion
Meaning - It checks the state of L1 cache with that of DB , fires DML suitably
In case of new entity -insert
IN case of updated state - update
In case of removed entity - delete
3. session.close() -
L1 cache is destroyed
DB cn rets to DBCP

Q - What exactly happens upon rollback ?(tx.rollback)
only step 3 will be performed.

Today's Topics
1. Entity States

transient , persistent , detached

Triggers for persistent -> detached
1. session.close()
2. Session API
2.1 public void evict(Object persistentObj) throws HibernateException
Detaches specified persistent entity from the L1 cache.
2.2 public void clear() throws HibernateException
Removes all persistent entities from the L1 cache.
(L1 cache is not destroyed n db cn doesn't return to the pool)

2. Objective : To get all user details
Can you solve it using session.get -

3. Solve it using HQL(Hibernate query language)/JPQL (Java Persistence Query Language)

Object oriented query language, where table names are replaced by POJO class names & column names are replaced by POJO property names, in case sensitive manner n ? replaced by named IN parameters.
3.1 sql - select * from users
HQL - from User
JPQL - select u from User u
u - alias for entity

3.2 Create Query Object , from Session i/f
public <T> org.hibernate.query.Query<T> createQuery(String jpql/hql,Class<T> resultType)
T --result type.

3.3 To execute select query
Query i/f method
public List<T> getResultList() throws HibernateException
-Rets list of PERSISTENT entities.

4. Objective : Display all users born between strt date n end date & under a specific role
I/P : begin dt , end date , role

4.1 To Pass named IN params to the query
Query i/f method
public Query<T> setParameter(String paramName,Object value) throws HibernateException.

5. User Login (Lab work)
i/p : email n password
o/p User details with success mesg or invalid login mesg

6. Objective : Display all user's last names under a specific role
JPQL - ???
Steps - ??

7. Objective : Display all users first name , last name,dob under a specific role
JPQL - ???
Steps - ??

String jpql ="select u.firstName,u.lastName,u.dob from User u where u.role=:rl"

List<Object[]> list=session.createQuery(jpql,Object[].class).setParameter("rl", userRole).getResultList();

In Tester :
list.forEach(o -> sop(o[0]+" "+o[1]+" "+o[2]));

In case of fetching multiple properties from the DB ,
use a JPQL constructor expression

eg :
jpql = "select new com.sunbeam.entities.User(firstName,lastName,dob) from User u where u.role=:rl"
What will be type of List<User>
Pre requisite : MATCHING constructor in POJO/Entity class

8. Update

1. Change password
i/p --email , old password , new pass
o/p : mesg indicating success or a failure

Use - Query API

public <T> T getSingleResult()

Executes a SELECT query that returns a single PERSISTENT result.

Throws:
NoResultException - if there is no result
NonUniqueResultException - if more than one result
IllegalStateException - if called for a JPQL UPDATE or DELETE statement

Session API
public void clear()
Detaches ALL the entities from L1 cache

9.Bulk update -
Apply discount to reg amount , for all users , dob before a specific date

i/p -- discount amt, date

String jpql="update User u set u.regAmount=u.regAmount-:disc where u.dob < :dt";

9.1 Create a Query (DML - update | delete)
Session API
public Query<T> createQuery(String jpql) throws HibernateException
jpql -- DML(update | delete)

9.2 Query API
public int executeUpdate() throws HibernateException
--use case --DML

10. Un subscribe user
i/p user id
o/p user details removed from DB
Steps
get user (persistent) ref from it's id - session.get
null checking - yes - give err mesg
not null - session.delete(ref)

11. Lab work
Delete records for specified role
i/p - role
o/p mesg
jpql="delete from User u where u.userRole=:rl"

executeUpdate

12.. Save n restore images to / from DB
Instead of using FileInputStream n FileOutputStream ,use simpler API
Use FileUtils class - from Apache supplied - commons-io.jar

```
<!-- https://mvnrepository.com/artifact/commons-io/commons-io -->
  <dependency>
   <groupId>commons-io</groupId>
   <artifactId>commons-io</artifactId>
   <version>2.11.0</version>
  </dependency>
```

Methods of FileUtils class

1. public static byte[] readFileToByteArray(File file)
                                throws IOException
Reads the contents of a file into a byte array. The file is always closed, after data read.

2. public static void writeByteArrayToFile(File file,
                        byte[] data)
                                throws IOException
Writes a byte array to a file creating the file if it does not exist.

13. save vs persist vs saveOrUpdate vs merge